Kairos Autonomi
498 W. 8360 S.
Sandy, Utah 84070
801-255-2950 (office)
801-907-7870 (fax)
www.kairosautonomi.com

BULLETIN
BUL-110

Overview

This document will walk the reader through the full SharedLink protocol, an initial implementation of SharedLinks critical protocol features and then the usage of SharedLink and its tunneled pipe commands in a TeleOp Only sub-set of the full Pronto4 system.

These referenced documents provide additional information for a more detailed understanding of the full Pronto4 system including its shared variable and pipe structures.

Documents

SharedLinkF Configuration, 1.00.04	
Kairos Operating System (KairOS), V01_04_01_02	Bulletin 026
Pronto4 Shared Variables,Scripts, Protocols and Formats, V1.00.08	
ServoPod Actuator Control ICD	
Pronto4 Pipe Structure	
Kairos Autonomi Source Code Overview	
SharedLink UDP Beacon, V01.00.01	Bulletin 009

Table of Contents

SharedLink Protocol 2
Limited SharedLink Implementation 10
Teleop Only SharedLink Implementation 13
Shared Variables Referenced 13
Inbound SharedLink Shared Variable List 15
Outbound SharedLink Shared Variable List 15
Teleop Only Pipe Commands 21
Teleop Only Sequencing 26
SharedLink Windows Application Example 28
Example SharedLink Communications Stream 31

Kairos Autonomi
498 W. 8360 S.
Sandy, Utah 84070
801-255-2950 (office)
801-907-7870 (fax)
www.kairosautonomi.com

SharedLink Protocol

The SharedLink protocol uses a series of fields contained within framing characters. Each of the fields are defined by a command character. The protocol uses on ASCII values and is case sensitive. Any character that is outside of the standard printable ASCII character set from 33 to 126 decimal

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

The following characters are used to signify various fields in the SharedLink protocol. They are case sensitive and their order of usage in the message frame is important.

[— Comm channel open
: — Address field
Q — Query Field
R — Response Field
E — Remote pipe Field
D — Data Field
N — Ordinal data fields, me to you, Mode 2
L — Ordinal data fields, you to me, Mode 1
C — Checksum Field
T — Type Field
M — Mode field
P — Ping Field
G — Pong Field
A — All field, Mode 3
| — Field break/delimiter (pipe character)
] — Comm channel close
crlf — Carriage return, linefeed (optional)

Channel Control

[Opens a communications channel and indicates that field characters are expected

] Closes a communications channel and indicates only an Open channel character is acceptable

Field Break

| Used to delimit or terminate a field.

Address Field

:d| — Address field, destination element supplied
d = A - W, A=1, X=Global
:ds| — Address field, destination and source elements supplied
d = A - W, A=1, X=Global
s = A - W, A=1, X=Global
Extra characters after source element but before field break are ignored and discarded

Query Field
Based on previously send mode of shared link
Mode 0 Shared variable named query mode
Qsvname| — Query specific shared variable tag name
svname — name of shared variable
Mode 1 Outbound shared variable list population mode
Qsvname| — Specific shared variable tag name to add to Outbound list
svname — name of shared variable
Mode 2 Inbound shared variable list population mode
Qsvname| — Specific shared variable tag name to add to Inbound list
svname — name of shared variable
Mode 3 All shared variable mode
— Ignored

Data Field
Mode 0 Basic shared variable update mode, shared variable name supplied
— Inbound list searched for shared variable, updated if exists
— List is marked if shared variable does not exist
Mode 1 Outbound shared variable ordinal list mode
— Data ignored
Mode 2 Inbound shared variable ordinal list mode
— Data written to indexed shared variable
Mode 3 All shared variable mode
— Data written to supplied shared variable
— Shared variable created if needed

Response Field
Rsvfield — Response for previous query of shared variable tag name

Remote Pipe Field
Epipefield — Name of remote pipe to send command specified in the Data Field

Data Field
Dvalue — Value to associate with supplied shared variable or list position

List Fields
Lord — Ordinal data fields, you to me, Mode 1
— ord is offset into list of this field
Nord — Ordinal data fields, me to you, Mode 2
— ord is offset into list of this field

Aord — All field, Mode 3
— ord is offset into list of this field

Checksum Field
Cvalue — The checksum of the fields from address field through to C
A modulo 256 hex value of the ASCII character additions

Type Field
Ttype — Type field of shared variable tag following the type field.
Type = 0 – long, 2 – double, 4 – string

Mode Field
Moptions — Sets the mode protocol operation on remote machine.
UI — Inbound List is Unlocked
LI — Inbound List is Locked
UO — Outbound List is Unlocked
LO — Outbound List is Locked
L — All lists are locked
R — Read from INI file
W — Write lists to INI file

Ping Field
Pmsg — Send a Ping out to the remote machine, round trip is measured
Msg is a token that is expected back

Pong Field
Gmsg — Send a Pong back to the host machine
Msg is the received Ping message

The following examples show how the messages of the protocol are assembled. Pay particular attention to the order of the fields in the messages.

Query Specific Shared Variable
[:sd|Qsvname|Dxxxx|Cnn] Query of shared variable packet

[Open communications channel
:sd| Address field with source and destination
Ttype Type of shared variable to respond with, Optional
— long = 0, double = 2, or string = 4
Qsvname Name of shared variable to query
— If variable does not exist, zero or null is returned
Cnn Checksum of entire packet from: to C
] Close communications channel

Response to Specific Query
[:sd|Rsvname|Dxxxx|Cnn] Response without type specified packet
[:sd|Ttype|Rsvname|Dxxxx|Cnn] Response with type specified packet

[	Open communications channel
:sd|	Address field with source and destination
Ttype	Type of shared variable to respond with, Optional — long=0, double=2, or string=4
Rsvname	Name of shared variable with data following — If variable does not exist it is created with type
Dxxxx	Data to write into shared variable, proper type context
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Send All Shared Variable Information Packet

[:sd|Aord|Ttype|Qsvname|Dxxxx|Cnn] Full shared variable packet

[	Open communications channel
:sd|	Address field with source and destination
Aord|	Set packet mode 3, all shared variables ord — not needed, ignored if supplied
Ttype|	Type of shared variable to respond with, Optional — long=0, double=2, or string=4
Qsvname|	Name of shared variable with data following — If variable does not exist it is created with type
Dxxxx|	Data to write into shared variable, proper type context
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Send Ordinal List, me to you

[:sd|Nord|Qsvname|Qsvname|…Cnn]
[:sd|Nord|Ttype|Qsvname|Ttype|Qsvname|…Cnn]

[	Open communications channel
:sd|	Address field with source and destination
Nord|	Set packet mode 2, OutBound list, you to me ord = 0 to n, Starting ordinal in data list
Ttype|	Type of shared variable to respond with, Optional — long=0, double=2, or string=4
Qsvname|	Name of shared variable with data following — If variable does not exist it is created with type
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Remote list may be locked. If so this packet is ignored

Send Ordinal List, you to me

[:sd|Lord|Qsvname|Qsvname|…Cnn]
[:sd|Lord|Ttype|Qsvname|Ttype|Qsvname|…Cnn]

[	Open communications channel
:sd|	Address field with source and destination
Lord|	Set packet mode 1, OutBound list, you to me ord = 0 to n, Starting ordinal in data list
Ttype|	Type of shared variable to respond with, Optional — long=0, double=2, or string=4
Qsvname|	Name of shared variable with data following — If variable does not exist it is created with type
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Remote list may be locked. If so this packet is ignored

Send Ordinal Data, me to you

[:sd|Nord|Dxxxx|Dxxxx|Dxxxx|Dxxxx|…Cnn]
[:sd|Nord|Dxxxx|D|D|Dxxxx|…Cnn]

[	Open communications channel
:sd|	Address field with source and destination
Nord|	Set packet mode 2, OutBound list, you to me ord = 0 to n, Starting ordinal in data list
Dxxxx|	Data to write into shared variable, proper type context
D|	Data did not change, ordinal place holder
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Ping Packet

[:sd|Pmsg|Cnn] Ping query packet

[	Open communications channel
:sd|	Address field with source and destination
Pmsg|	Ping field with unique message
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Pong Packet

[:sd|Gmsg|Cnn] Pong response packet

[	Open communications channel
:sd|	Address field with source and destination
Gmsg|	Pong field with reflected Ping message
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Request Outbound List

[:sd|N?|Cnn] Outbound list request

[	Open communications channel
:sd|	Address field with source and destination
N?|	Request outbound list
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Request Inbound List

[:sd|L?|Cnn] Inbound list request

[	Open communications channel
:sd|	Address field with source and destination
L?|	Request inbound list
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Request All Shared Variables

[:sd|A?|Cnn] All shared variable request

[	Open communications channel
:sd|	Address field with source and destination
A?|	Request all shared variables
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Unlock Inbound List

[:sd|MUI|Cnn] Unlock inbound list

[	Open communications channel
:sd|	Address field with source and destination
MUI|	Unlock inbound list command
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Unlock Outbound List

[:sd|MUO|Cnn] Unlock outbound list

[	Open communications channel
:sd|	Address field with source and destination
MUO|	Unlock outbound list command
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Lock Inbound List

[:sd|MLI|Cnn] Lock inbound list

[	Open communications channel

:sd|	Address field with source and destination
MLI|	Lock inbound list command
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Lock Outbound List

[:sd|MLO|Cnn] Lock outbound list

[	Open communications channel
:sd|	Address field with source and destination
MLO|	Lock outbound list command
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Lock All Lists

[:sd|ML|Cnn] Lock all lists

[	Open communications channel
:sd|	Address field with source and destination
ML|	Lock all lists command
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Write Lists to INI file

[:sd|MW|Cnn] Save list to INI file

[	Open communications channel
:sd|	Address field with source and destination
MW|	Write lists to INI file command
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Read Lists from INI file

[:sd|MR|Cnn] Read list from INI file

[	Open communications channel
:sd|	Address field with source and destination
MW|	Read lists from INI file command
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Send pipe command

[:sd|Epipename|Dxxx|Cnn]

[	Open communications channel
:sd|	Address field with source and destination
Epipename|	Remote pipe name

Dxxx|	Remote pipe data
Cnn	Checksum of entire packet from : to C
]	Close communications channel

Limited SharedLink Implementation

The SharedLink protocol is pretty straightforward to implement initially. There are a number of other fields that add to the complexity. Initially we will just have you implement the basic functionality

SharedLink consists of two lists that are shared between two computers. It is primarily a point to point protocol but does have some addressing capability.

The protocol is a message format with open and close characters and line termination characters.

'[' Open channel character
']' close channel character
crlf End of line character pair

Shared link messages are contained between the open and close characters as shown.

[header addressing messagebody] crlf

The header is a ':' character and the addressing is the character 'A' through 'P' assigned to address 1 - 16

The destination character is first followed by the source character.

[: A B messagebody] crlf

This message is addressed to node 1 sent from node 2. Sent to A from B

Fields within the message body are separated by the pipe character "|"

The messagebody begins with a pipe character and ends with the close channel character "]"

The messagebody consists of many fields based upon the mode of the packet. The message body ends with a checksum field.

Each field begins with a field type character 'D' for Data, 'C' for checksum etc.

Each field ends with the next pipe character "|" or the close channel character']'

It is assumed that all valid messagebodies end with a checksum field.

[: A B |message fields| C xx] crlf

'xx' is the calculated checksum

As mentioned, the message fields are basically mode sensitive based upon the sequence of messages.

The checksum is a simple modulo 256 addition of the ASCII values of all the characters from the header ":" to the checksum field indicator "C" inclusive. The following pseudo formula indicates the checksum approach

(ASCII(:) + ASCII(A) + ASCII(B) + ASCII(|) + ... + ASCII(|) + ASCII(C)) MODULO 256

Initially it is easiest to assume that the lists are known a priori. That is that both ends of the protocol are already aware of the inbound and outbound shared variable list.

Each end of the serial stream or node sends a SharedLink message consisting of a 'D' field for each Shared Variable in the outbound list.

The message body starts out with an 'N' field that indicates what shared variable 'Set' is being transmitted. It is safe to always an 'N' field with a value of 0 initially. THis is used to break up long lists of shared variables into smaller messagebody sizes.

[: A B | N 0 | messagebody | C xx] crlf

The spaces shown are for clarity only. The actual SharedLink message does not have any spaces placed intentionally, unless part of a message body

The 'D' or data field contains the string representation of the value of the Shared Variable assigned to that position of data fields. There is a single data field for each shared variable in the outbound shared variable list

Assume the outbound SharedVariable list has four(4) entries as:

AutoGpsLat
AutoGpsLon
AutoGpsHead
AutoGpsVel

The simple message to send would be:

[: A B | N 0 | D 40.12345678 | D -113.12345678 | D 85.6 | D 22.1 | C xx] crld

Whereas: AutoGpsLat = 40.12345678 degrees
AutoGpsLon = -113.12345678 degrees
AutoGpsHead = 85.6 degrees
AutoGpsVel = 22.1 mph

The other end of the SharedLink connection would receive this message into the same SharedVariable list as the inbound shared variable list.

Initially assume that both sides know what the inbound and outbound lists of each other are. The Inbound and Outbound list on a node do not need to be the same. The Inbound list on node 'A' needs to be the same as the outbound list on node 'B' and visa versa.

This is a very simple method and some of the simplest usages of the shared linkprotocol.

For complete detail please check the SharedLink specification. Additional elements are as follows.

If the 'D' has not changed from the last transmission, the value can be eliminated to save bandwidth. This is called shared variable compression.

If shared variable compression is used, then the full data must transmitted periodically even though it has not changed, usually transmitted once per second.

The actual name of the shared variable can preceed the 'D' field by adding an 'R' field. The 'R" field indicates which the shared variable the following 'D' field value belongs too.

The 'Q' field requests that the following shared variable name is returned as populated 'R' field and 'D' fields. This is the ability to request and respond with specific adhoc shared variables.

Teleop Only SharedLink Implementation

The entire Pronto4 system consists of many commands that are not available in all systems based upon complexity of that system.

The following is the minimum of commands required to implement a TeleOp system.

The following Shared Variable are created in a TeleOp only system and used for control and data monitoring.

Shared Variables Referenced

AutoGpsHead	double	Vehicle GPS heading
AutoGpsLat	double	Vehicle GPS latitude
AutoGpsLon	double	Vehicle GPS longitude
AutoGpsVel	double	Vehicle GPS velocity
AutoGpsReferences	double	Vehicle GPS satellites
AutoGpsQuality	double	Vehicle GPS quality, 0-100%
AutoGpsStamp	double	Vehicle GPS time stamp
gps_count	integer	Incrementing count of GPS packets received
rx_link_count	integer	Incrementing count of SharedLink messages received
tx_link_count	integer	Incrementing count of SharedLink messages transmitted
rx_link_rate	integer	Measured rate in ms that SharedLink messages are received
tx_link_rate	integer	Measured rate in ms that SharedLink messaged are transmitted
video_mux1	integer	Channel of the multiplexed video selected
veh_start	integer	Start vehicle, 1=start
veh_enable	integer	Enable vehicle 1=enable, 0=disable
veh_motion	integer	Motor motion enabled 1=enabled, 0=disabled
joy_steer	integer	Desired position in degrees of the steering wheel
veh_steer	integer	Actual position in degrees of the steering wheel
veh_brake	integer	Desired position 0-100% of the brake
veh_throttle	integer	Desired position 0-1000 (0-100.0%) of the throttle
veh_shift	integer	Desired position of Shift (0-Park,1-Rev,2-Neu,3-Drive)
pause	integer	Vehicle pause state, 1=pause
key_on	integer	Vehicle Key on state, 1=KeyOn
eng_enb	integer	Enable command, 1=enabled
pe7	integer	EStop/Manual state, 1=EStop/Manual
bbrake	integer	Commanded brake position
fbrake	integer	Commanded transmission position
steerang	integer	Actual Steering angle in degrees
throttle	integer	Commanded throttle position
svp_enb_eng	integer	Actual enable state, 1=enabled
serialpck	integer	Incrementing of SVP packets received
veh_gear	integer	Actual vehicle gear, 0-Shifting, 1-Prk, 2-Rev, 3-Neu, 4-Drv
veh_state	integer	Operational state of vehicle

veh_estop	integer	EStop state of vehicle
veh_pause	integer	Pause state of vehicle
veh_obstacle	integer	Obstacles detected
vehicle_bat	double	Vehicle battery voltage in volts
run_composite	string	Detailed run information about vehicle
VehicleName	string	Name of vehicle
Health_total	integer	Quality of vehicle from running tests
rpm	integer	General RPM value, relatively correcty
fuel	integer	Fuel percentage when available
sys_thd	integer	System harddrive temperature when available
svp_reboot	integer	Number of times ServoPod has been rebooted
id_rx_count	integer	Incrementing count of ID receptions
id_tx_count	integer	Incrementing count of ID transmissions
jauslp_count	integer	Incrementing count of JAUS ID receptions
svbeacon_count	integer	Incrementing count of SV beacons transmissions
estop_rx_count	integer	Incrementing count of ESTOP receptions
estop_tx_count	integer	Incrementing count of EStop transmissios

JAUS
These variables are available in systems that use JAUS as the communications protocol rather than SharedLink.

jausrx_count	integer	Incrementing count of JAUS message receptions
jaustx_count	integer	Incrementing count of JAUS transmissions
obd_rpm	integer	RPM derived from OBDII port
obd_odo	integer	Odometer derived from OBDII port
obd_engine	double	Engine temperature derived from OBDII port
obd_fuel	double	Fuel level derived from OBDII port
obd_speed	double	Vehicle Speed derived from OBDII port

Inbound SharedLink Shared Variable List

These are the Shared Variables that are received from the OCU that cause the vehicle to perform actions as desired in TeleOp mode.

video_mux1	Select video multiplexor channel, usually just 1 channel
veh_start	Start Engine, 1=start
veh_enable	Enable vehicle, 1=Enable
veh_motion	Enable motion or Deadman active, 1=enable
joy_steer	Steering Command, +-450
veh_steer	
veh_brake	Vehicle brake command, 0-100%
veh_throttle	Vehicle throttle command, 0-1000 (0-100.0%)
veh_shift	Vehicle shift (0-Park, 1-Rev, 2-Neu, 3-Drv)

Outbound SharedLink Shared Variable List

These are the shared variables that are transmitted from the vehicle to the OCU as status and feedback.

AutoGpsHead	Vehicle GPS heading
AutoGpsLat	Vehicle GPS latitude
AutoGpsLon	Vehicle GPS longitude
AutoGpsVel	Vehicle GPS velocity
AutoGpsReferences	Vehicle GPS satellites
AutoGpsQuality	Vehicle GPS quality, 0-100%
AutoGpsStamp	Vehicle GPS time stamp
pause	Pause state, 1=pause
key_on	Keyon State, 1=keyon
eng_enb	Engine Enable state, 1=enable
pe7	Estop/Manual state, 1=EStop/Manual
bbrake	Commanded brake position
fbrake	Commanded transmission position
steering	Commanded steering position in ticks
throttle	Commanded throttle position
svp_enb_eng	Alternate enabled state of ServoPod
serialpck	Count of ServoPod serial messages
veh_gear	Vehicle gear achived, 0-Shifting, 1-Park, 2-Rev, 3-Neu, 4-Drv
veh_state	Vehicle operational state
veh_estop	In cab Estop, 1=EStop, 0=No Estop
veh_obstacle	vehicle obstacles, if available
vehicle_bat	vehicle battery voltage
run_composite	Detailed run state of vehicle, if available
VehicleName	Name of vehicle
Health_total	Health of system, 0-100%, if available

rpm	Vehicle RPM, if available
fuel	Vehicle fuel level 0-100%, if available
sys_thd	Vehicle temperature, in degrees C, if available

The following are shared variable states that are assigned based upon the type of system and the software executing. This document concerns the Tele-Op only systems so only the Embedded C section is valid. The other sections are provided for informational purposes only.

Embedded C – RCM67xx

```
run_composite="Running"          Only State
Health_Total = 100
run_status      Not used
run_method      Not used
run_progress    Not used
run_teleop      Not used
veh_state       Not used
```

Windows Based – djIVN

```
Health_Total = Value from Built In Test Facility (0-100%)
run_composite = "{invalid}"                          Initial State
run_composite = "SharedLink Motion Enabled"          When motion enable is pressed
run_composite = "SharedLink Motion Disabled"
run_status      Not used
run_method      Not used
run_progress    Not used
run_teleop      Not used
veh_state       Not used
```

Windows Based Path Following System - djDriveWB

```
Health_Total = Value from Built In Test Facility (0-100%)
Vehicle State Table
        veh_state = 1          Ready
        veh_state = 2          Pause
                               Transition when RadioEstop detected
        veh_state = 3                RadioStop - Ready
        veh_state = 4                RadioStop - Pause
        veh_state = 5          Radio E-Stop


run_composite = {invalid}      Fire-up state

Teleop
    run_composite.value =      run_method,
                               ,
                               run_status,
                               run_progress,
                               Waypoint:  run_wp
PathFollowing
    run_composite.value =      run_method,
                               Teleop: run_teleop,
```

	run_status,
	run_progress,
	Waypoint: run_wp
run_method = "{invalid}"	Fire-up state
run_method = “Teleop”	Tele-Operation
run_method = “Path”	Path Following, Single Course
run_method = “SinglePt”	Single Point Achievement
run_method = “Multi”	Multiple Course – Ingress, Loop, EGress
Modifiers	
“, Drv”	Forward Driving
“, Rev”	Reverse Driving
“, Fwd”	Beginning of course to End of Course
“, Bck”	End of Course to Beginning of course
“, Cur”	Start at current position
“, Beg”	Start at beginningof course
“, @xxxmph”	At Fixed speed, xxx MPH
run_status = “{invalid}”	Fire-up state
run_status = “Aborted”	Course Aborted
run_status = “Complete”	Course Complete
run_status = “Running”	Course path followingin operation
run_status = "RadioPause"	Course paused by Radio input
run_status = "UserPause"	Course paused by local user
run_status = "E-Stop"	E-Stop Condition
run_status = "Resume"	Course Resumed
run_status = "Idle"	No Course in operation, waiting
run_progress=”{invalid}”	Fire-up state
run_progress=””	No pathing active
run_progress=”Ingress”	Executing Ingress Course
run_progress=”Loop”	Executing Loop Course, first loop
run_progress=”Egress”	Executing Egress Loop
run_progress=”LoopWait”	Loop is paused
run_progress=”LoopExit”	Loop Exit has been initiated
run_progress=”Looping”	Looping course is executing, >1
run_progress=”LoopLast”	Last loop before exit
run_teleop = "{invalid}"	Fire-Up state
run_teleop = "Assisted"	Executing Assisted Tele-operation
run_teleop = "Seamless"	Executing Seamless Tele-Operation
run_teleop = “None”	No Tele-Operation state selected

EStop Beacon Messaging

The EStop beacon system consists of three UDP packets that are transmitted from the EStop source to the EStop Receivers. Unmanned Ground Vehicles are usually EStop Receivers and OCUs or Independent EStops are considered EStop sources.

EStop Messages are broadcast to 255.255.255.255 to port 7000

These rules apply for the EStop Source:

1. Transmit the message 1 per second
2. Select the message to transmit based upon the status of the EStop switches

EStop Active	Send EStop Message
EStop InActive, Pause Active	Send Pause Message
EStop InActive, Pause Inactive	Send Run Message

3. Send message immediately upon switch state change.

These rules apply for the EStop Receiver:

1. If no message received for 3 seconds enter EStop State
2. If EStop message received, immediately enter EStop State
3. While in EStop State if Pause message received, wait 5 seconds, enter Pause State
4. While in EStop State if Run message received, wait 5 seconds, enter Run State
5. While in Run State if Pause message received, enter Pause State Immediately
6. While in Pause State if Run message received, wait 5 seconds, enter Run State

The following arrays contain the messages for each of the three message types – Stop, Pause and Run. These messages are historical in nature for deployed systems. They are shown in both Hex and Decimal.

Safety_EStop = Array(36, 40, 178, 170, 17, 39, 162, 11, 253, 223, 0, 0, 85, 170, 0, 0, 42, 42, 68, 79, 90, 69, 82, 95, 66, 65, 83, 69, 42, 42, 0, 0, 0, 0, 0, 0, 0, 93, 13, 10)

Safety_Pause = Array(36, 40, 178, 170, 17, 39, 162, 11, 253, 223, 0, 0, 170, 85, 0, 0, 42, 42, 68, 79, 90, 69, 82, 95, 66, 65, 83, 69, 42, 42, 0, 0, 0, 0, 0, 16, 1, 93, 13, 10)

Safety_Run = Array(36, 40, 178, 170, 17, 39, 162, 11, 253, 223, 0, 0, 170, 85, 0, 0, 42, 42, 68, 79, 90, 69, 82, 95, 66, 65, 83, 69, 42, 42, 0, 0, 0, 0, 0, 0, 0, 93, 13, 10)

```
estp_stop_pkt[] = {
                  //    0    1    2    3    4    5    6    7    8    9
                  /*00*/ 0x24,0x28,0xB2,0xAA,0x11,0x27,0xA2,0x0B,0xFD,0xDF,
                  /*01*/ 0x00,0x00,0x55,0xAA,0x00,0x00,0x2A,0x2A,0x44,0x4F,
                  /*02*/ 0x5A,0x45,0x52,0x5F,0x42,0x41,0x53,0x45,0x2A,0x2A,
                  /*03*/ 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x5D,0x0D,0x0A};

estp_pause_pkt[] = {
                  //    0    1    2    3    4    5    6    7    8    9
                  /*00*/ 0x24,0x28,0xB2,0xAA,0x11,0x27,0xA2,0x0B,0xFD,0xDF,
                  /*01*/ 0x00,0x00,0xAA,0x55,0x00,0x00,0x2A,0x2A,0x44,0x4F,
                  /*02*/ 0x5A,0x45,0x52,0x5F,0x42,0x41,0x53,0x45,0x2A,0x2A,
                  /*03*/ 0x00,0x00,0x00,0x00,0x00,0x10,0x01,0x5D,0x0D,0x0A};

estp_run_pkt[] = {
                  //    0    1    2    3    4    5    6    7    8    9
                  /*00*/ 0x24,0x28,0xB2,0xAA,0x11,0x27,0xA2,0x0B,0xFD,0xDF,
                  /*01*/ 0x00,0x00,0xAA,0x55,0x00,0x00,0x2A,0x2A,0x44,0x4F,
                  /*02*/ 0x5A,0x45,0x52,0x5F,0x42,0x41,0x53,0x45,0x2A,0x2A,
                  /*03*/ 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x5D,0x0D,0x0A};
```

ID Beacon Messaging

ID Beacon messages are broadcast from each robotic asset once every 5 seconds. This ID Beacon can be from a system that is dormant or active. Each field in the message is delineated by a pipe character '|' . There is no checksum as it is enclosed in a UDP packet. The ID Beacon is broadcast to 255.255.255.255 on both Port 1901 and 1902.

Example ID Beacon message from VEH_SN102Nissan:

SN11110334|192.168.200.102|VEH_SN102Nissan|Vehicle SH/JAUS/IVN|djBasis.exe, djEtherMap.exe, djPinger.exe, djJAUS_IVN.exe, DJ_LDR.EXE, djSharedLCD.exe|40.59970333|-111.90721333|325.34|4.1|||03-26-2013|07:38:21|Front/RearVideo,192.168.200.172|

Example ID Beacon meassage from OCU

OCULaptop1|127.0.0.1|SIM_SN104GATLaptop2|Shepherd_OCU|None|0|0|0|0|Monitor2|127.0.0.1:24800|10-18-2013|16:03:05|Video,192.168.200.232;VIM,192.168.200.100|

Modified Example for Embedded CPU

RCM6700|127.0.0.1|VEH_SN00001|Vehicle_SH/SL/IVN|None|0|0|0|0|None|0.0.0.0|10-18-2013|16:03:05|Asset1,0.0.0.0;Asset2,0.0.0.0|

'Name and function of each beacon field, Beacon fields are "|" pipe separated

HostName	Name of computer support this robot asset
IP Address	IP address of the robotic asset
VehicleName	Name of vehicle or asset
DJBasis Set	Program Set loaded
ExecutableList (comma separated)	List of programs loaded and running
Latitude	Decimal Latitude of asset
Longitude	Decimal Longitude of asset
Heading	Heading of asset in degrees
Velocity	Velocity of asset in mph
SynergyHost	Host for multiple assets
SynergyIP	IP for multiple asset host
Start Date	Fire-up time
Start Time	Fire-up date
EthernetAssetList (Name1,IP1;Name2,IP2)	Name and IP of available ENet assets

The following pseudo code example shows an easy way yo create a beaconing system for usage with Pronto4 series robotic systems. The ID packet is built and then transmited. It is human readable so its contents can be discerned in network analysis tools like WireShark.

```
'Pseudo Code Usage Example
        BeaconStart("");                          //Start Beacon with text or clear buffer
        BeaconAdd("RCM6700");                     //Computer Name, prepend with pipe "|"
        BeaconAdd("192.168.200.201");             //Computer IP
        BeaconAdd(ReadSV("VehicleName");          //Vehicle Name
        BeaconAdd("Vehicle_SH/SL/IVN");           //Vehicle Configuration
        BeaconAdd("None");                        //Program Execution List
        BeaconAdd(ReadSV("AutoGpsLat");           //Vehicle Latitude
        BeaconAdd(ReadSV("AutoGpsLon");           //Vehicle Longitude
        BeaconAdd(ReadSV("AutoGpsHead");          //Vehicle Heading
        BeaconAdd(ReadSV("AutoGpsVel");           //Vehicle Velocity
        BeaconAdd("None");                        //Synergy Host Name
        BeaconAdd("None");                        //Synergy IP
        BeaconAdd("StartDate");                   //Vehicle Fire up Date
        BeaconAdd("StartTime");                   //Vehicle Fire up Time
        BeaconAdd("Asset1,0.0.0.0;Asset2,0.0.0.0")// Ipassets available
        SendIP(BeaconEnd)

BeaconStart        //Initialize Beacon Buffer
BeaconAdd          //Add field to beacon buffer and the append with "|"
BeaconEnd          //Return pointer to beacon string
```

Packet Breakdown

SN11110334|	Host Name
192.168.200.102|	IP Address of asset
VEH_SN102Nissan|	Name of Asset
Vehicle SH/JAUS/IVN|	Vehicle Configuration set
djBasis.exe, djEtherMap.exe, djPinger.exe, djJAUS_IVN.exe, DJ_LDR.EXE, djSharedLCD.exe|	Comma delimited list of running executables
40.59970333|	Latitude
-111.90721333|	Longitude
325.34|	Heading
4.1|	Velocity in MPH
|	unused
|	unused
03-26-2013|	Date asset powered up
07:38:21|	Time asset powered up
Front/RearVideo,192.168.200.172|	List of Ethernet Assets

Standardized IP Ports

These ports are used by the Pronto4 system to allow our robotic system to communicate with each other. Not all are required or used by the TeleOp only systems but are included here for completeness.

ID Beacon	Broadcast	1901, 1902
EStop Beacon Source	Broadcast	7000, 7001 only for Kairos Isolated EStop
ESTop Beacon Receiver	Broadcast	7000, 7001
JAUS Port	various	3794
SharedLink OCU Tx	Unicast	4000
SharedLink OCU Rx	Unicast	4xxx where xxx is last octet of IP
VIM Beacon	Broadcast	7003
VIM Receiver	Unicast	7004
SV Data Beacon	Broadcast	7005, 7013 are common
SV Beacon Source Pipe Cmd	Unicast	7006
Tower Beacon	Broadcast	7007
Tower Commands	Unicast	7008, 8000

Teleop Only Pipe Commands

Pipe command are tunneled to the SharedLink protocol to directed remote pipe end points. Each pipe command has a remote pipe that it is directed to and a data set. The data set consists of a tag value identifying the data type followed by the data.

These commands are the subset that is required for TeleOp only operation. Additional commands may or may not be supported. In larger systems the ServoPod pipe endpoint is exchanged for the DRIVE end point, but the commands are the same.

Remote Pipe	Data Set (tag, data)	
Link	0,setip {ip}	
Link	0,outbound {on|off}	
Link	0,inbound {on|off}	
Servopod	108	dSvpTogEnbEngine
ServoPod	122, START_ON	dCommand to ServoPod command line
ServoPod	122, {actuator command}	dCommand to ServoPod command line
ServoPod	0, Teleop {start|stop}	
ServoPod	0, drive {value}	
ServoPod	0, trans {value}	
ServoPod	0, {other sent to vehicle command line}	

Teleop Only Shared Variable Operation

The following graphic shows how the inbound and outbound lists are assigned to each other. The rate of update is nominally at 40hz in each direction for shared variable operations. Pipe operations occur on a deman basis so their frequency of update is variable based upon the application.

Shepherd OCU Side (A) — Vehicle/Robot Side (B)

Variables Written Locally

Scanned at 25ms Rate
SV only Sent on Change
All SV updated 1 sec
SV Order in list is critical
SV Spelling/Case is critical

OutBound SV List
video_mux1
veh_start
veh_enable
veh_motion
joy_steer
veh_steer
veh_brake
veh_throttle
veh_shift

UDP Packets →

Inbound SV List
video_mux1
veh_start
veh_enable
veh_motion
joy_steer
veh_steer
veh_brake
veh_throttle
veh_shift

Variables Read Locally

Variables Read Locally

Ex:
Read AutoGpsVel=30.2

InBound SV List
AutoGpsHead
AutoGpsLat
AutoGpsLon
AutoGpsVel
AutoGpsReferences
AutoGpsQuality
AutoGpsStamp
pause
.....
svp_enb_eng
serialpck
veh_gear
veh_state
veh_estop
veh_obstacle
vehicle_bat
run_composite
VehicleName
Health_total
rpm
fuel
sys_thd

← UDP Packets

OutBound SV List
AutoGpsHead
AutoGpsLat
AutoGpsLon
AutoGpsVel
AutoGpsReferences
AutoGpsQuality
AutoGpsStamp
pause
.....
svp_enb_eng
serialpck
veh_gear
veh_state
veh_estop
veh_obstacle
vehicle_bat
run_composite
VehicleName
Health_total
rpm
fuel
sys_thd

Variables Written Locally

Scanned at 25ms Rate
SV only Sent on Change
All SV updated 1 sec
SV Order in list is critical
SV Spelling/Case is critical

Ex:
Write AutoGpsVel=30.2

Variables Read/Written Locally, Not sent over UDP

Local SV List
GPS_Count
Tx_Link_Rate
.....

Local SV List
GPS_Count
Tx_Link_Rate
.....

Variables Read/Written Locally, Not sent over UDP

Receive Pipe Queue:
[:BA|EServoPod|D0,TELEOP START|C02]
[:BA|EServoPod|D0,TELEOP STOP|CBA]

← UDP Packets

Send Pipe Message:
[:BA|EServoPod|D0,TELEOP START|C02]

Send Pipe Message:
[:BA|EOCU|D0,Command 2|Cxx]

UDP Packets →

Receive Pipe Queue:
[:BA|EOCU|D0,Command 2|Cxx]
[:BA|EOCU|D0,Command 1|Cxx]

Teleop Only Sequencing

The following sequencing will place a Pronto4 system using SharedLink into a teleoperation mode and then begin teleoperation functions.

Identify Remote system

Each Pronto4 system beacons its ID with an ID Beacon command. This allows for multiple systems to be located and operated with a single OCU. It is possible to identify and select the remote Pronto4 system with the contents of this beacon message. The details of this beacon message are not part of this document but the beacon message is shown below for reference.

Example ID Beacon message from VEH_SN102Nissan:

SN11110334|192.168.200.102|VEH_SN102Nissan|Vehicle SH/JAUS/IVN|djBasis.exe, djEtherMap.exe, djPinger.exe, djJAUS_IVN.exe, DJ_LDR.EXE, djSharedLCD.exe|40.59970333|-111.90721333|325.34|4.1|||03-26-2013|07:38:21|Front/RearVideo,192.168.200.172|

As can be seen, the IP address of the robotic asset is included in the ID beacon as well as the capabilities of the robot, its GPS location, running programs, etc.

Additionally, it is possible to simply know the IP address of the desired system and use that a priori information to perform the following sequences.

Set up Remote SharedLink receiver

All SharedLink systems listen on the same port. These systems boot with their SharedLink transmissions disabled, but they are listening. These pipe commands set up the remote SharedLink system for bidirectional transmission session with the proper IP addressing.

[:BA|ELink|D0, SetIP, 192.168.200.118:4104, 4000|C1C]
[:BA|ELink|D0, outbound, on|CA0]
[:BA|ELink|D0, inbound, on|C1F]

[:BA|MUI|CE3]
[:BA|MUO|CE9]

[:BA|N0|T0|Qvideo_mux1|T0|Qveh_start|T0|Qveh_enable|T0|Qveh_motion|T2|Qjoy_steer|T0|Qveh_steer|T0|Qveh_brake|T0|Qveh_throttle|T0|Qveh_shift|Qveh_clutch|CC9]

[:BA|L0|T2|QAutoGpsHead|T2|QAutoGpsLat|T2|QAutoGpsLon|T2|QAutoGpsVel|T0|QAutoGpsReferences|T0|QAutoGpsQuality|T2|QAutoGpsStamp|T0|Qpause|T0|Qkey_on|T0|Qeng_enb|T0|Qpe7|T0|Qbbrake|T0|Qfbrake|T2|Qsteerang|T0|Qthrottle|Qsvp_enb_eng|T0|Qserialpck|T0|Qve

h_gear|T0|Qveh_state|T0|Qveh_estop|T0|Qveh_obstacle|T2|Qvehicle_bat|T4|Qrun_composite|T4|QVehicleName|T0|QHealth_total|T0|Qrpm|T0|Qfuel|T2|Qsys_thd|C52]

[:BA|MW|C9C]
[:BA|ML|C91]

Send TeleOp Start Command

This tells the remote Pronto4 to enter teleoperation Mode. The command is sent through a SharedLink tunneled pipe command to a remote pipe endpoint.

[:BA|EServoPod|D0,TELEOP START|C02]

Set Shared Variables

Shared variables are set to the desired value and the remote system reads the changes in the shared variables and takes the appropriate action.

veh_throttle = desired percentage
veh_brake = desired percentage
veh_steer = desired percentage
veh_shift = desired gear

Send TeleOp Stop Command

This tells the remote Pronto4 to enter teleoperation Mode. The command is sent through a SharedLink tunneled pipe command to a remote pipe endpoint.

[:BA|EServoPod|D0,TELEOP STOP|CBA]

Terminate Shared Link Session

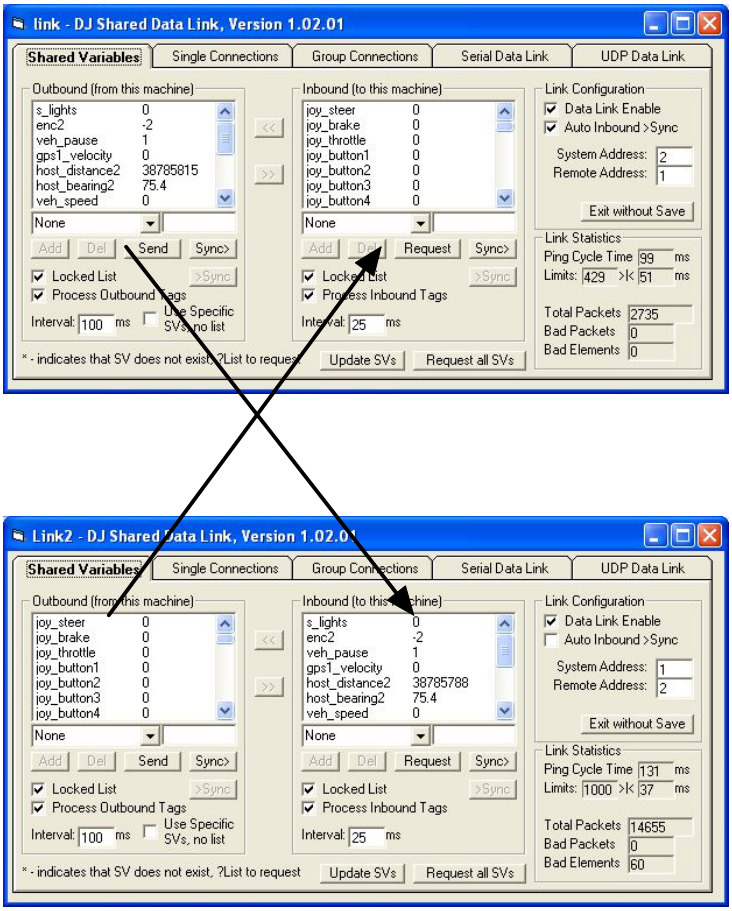
After a SharedLink session has been established and utilized it must be closed. These pipe commands terminate the session and restore the remote system to the listen only state.

Kairos Autonomi
498 W. 8360 S.
Sandy, Utah 84070
801-255-2950 (office)
801-907-7870 (fax)
www.kairosautonomi.com

SharedLink Windows Application Example

The SharedLink program transfers information from one computer to another. The information transferred is in the form of shared variables. Information is transferred over two media types — serial and UDP.

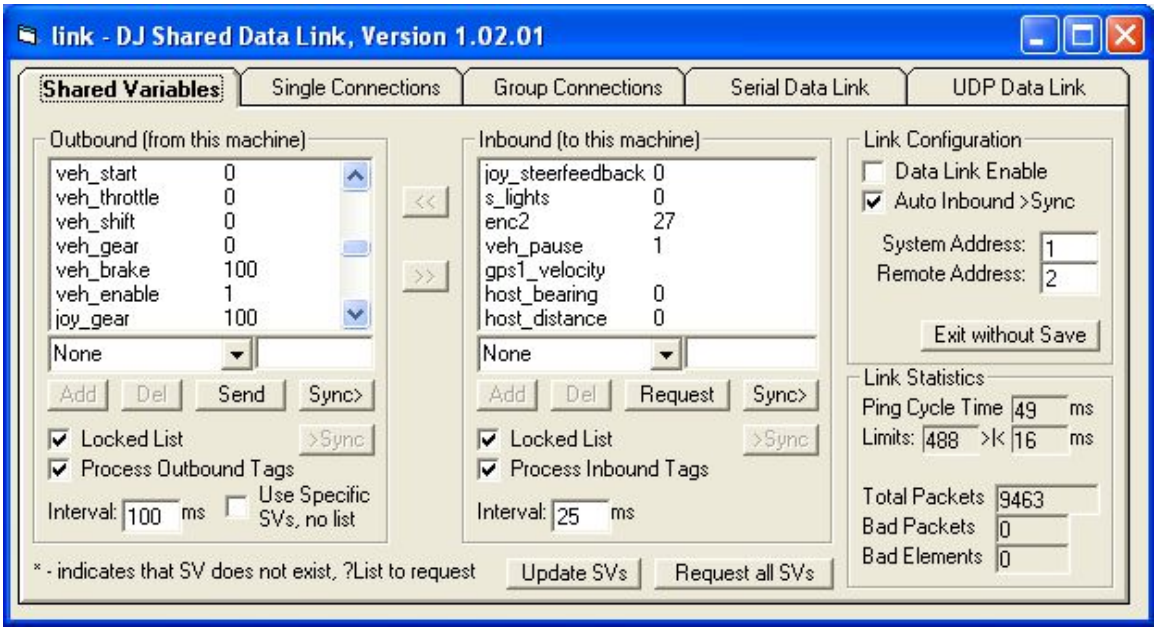
Transfer is accomplished in primarily a point to point fashion. It is possible to perform multi-drop transfers but then information is not bidirectional but from a source to many targets. The shared variables to transfer back and forth are set up in two lists. Each program is executed on a separate computer with an inbound and outbound list. The Inbound list of a target matches up with the outbound list of a source. Each computer has a target and a source or an inbound and outbound list.



The Outbound list on computer #1 contains a list of shared variables that are targeted at the inbound list on computer #2. It is crucial that the lists match on both computers. Tools and protocol features are supplied to assure that this occurs.

The Outbound list on computer #2 contains a list of shared variables that are targeted at the inbound list on computer #1.

Various command buttons and entry fields are disabled in the SharedLink program based upon the access control of the inbound and outbound lists. These lists are locked to help assure that they are not changed during the normal usage of the program. The default position is to have the access to these lists disabled.



ADD — Add the shared variable defined in the entry fields to the list.

DEL — Delete the selected shared variable from the list.

SEND — Perform a single send the value of the selected shared variable out over the media.

REQUEST — Perform a single request of the value of the selected shared variable.

Sync> — Send the shared variable names in the list to the corresponding list on the other computer. Inbound to outbound.

>Sync — Request the corresponding list of shared variables from the other computer. Inbound to outbound.

LOCKED LIST — Control the access state of the list. Shared variables can be used as variables but the names of the shared variables cannot be changed, added to or deleted.

PROCESS — Allow the list to be processed at the defined interval. The processing action takes place continuously.

UPDATE — Force an update of all the shared variables in the two lists. If the shared variables are not created then create them.

REQUEST ALL — Read all shared variables from the other computer and create/update them on this computer.

AUTO INBOUND SYNC — Perform an automatic synchronization of the inbound lists when the program is started.

SYSTEM ADDRESS — This is the address of this computer. A 0 is considered a global. Valid values are 1 to 20.

REMOTE ADDRESS — This is the address of the remote computer. A zero is considered a global. Valid values are 1 to 20.

Parameters entered into the SharedLink program are automatically saved to an INI file upon program exit and restored upon program execution.

Multiple executions of the SharedLink program are allowed. Each execution of the program uses a different INI file. It is possible and common to use multiple instantiations of the SharedLink program to share variables among several computers.

The SharedLink program also contains the ability to match shared variables to various requirements by performing single and grouped connections. In most cases these features are not needed and are not described in this document.

Example SharedLink Communications Stream

The following actual SharedLink messages show the full cycle of a teleoperation system. The System is initiated and changed into teleoperation mode. The vehicle is then manipulated after which teleoperation mode is stopped.

Sending out deltas with no change and periodic send of all data
Pings with no responses
No connection

```
....
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D0|D0|D0|D0|D0|D0|D100|D0|D0|D|C07]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D0|D0|D0|D0|D0|D0|D100|D0|D0|D|C07]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
....
```

Set IP address of remote system

```
<[:BA|ELink|D0, SetIP, 192.168.200.118:4104, 4000|C1C]

<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
```

Enable inbound and outbound lists

```
<[:BA|ELink|D0, outbound, on|CA0]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D0|D0|D0|D0|D0|D0|D100|D0|D0|D|C07]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|G0|C6F]
>[:AB|N0|D268.870025|D40.599864|D-
11.905448|D1.267880|D6|D70|D212912.390000|D0|D1|D1|D1|D|D|D0|D|D1|D23203|D|D|D|D0|D1.780000|D|D|D100|D0|D0|D85|C81]
<[:BA|ELink|D0, inbound, on|C1F]

<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D270.730010|D40.599864|D-
111.905448|D1.059080|D5|D70|D212912.609000|D0|D1|D1|D1|D|D|D0|D|D|D23204|D|D|D|D|D1.840000|D|D|D|D0|D|D|CD7]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D0.000000|D40.599864|D-
111.905448|D0.980200|D5|D70|D212912.796000|D0|D1|D1|D1|D|D|D0|D|D|D23205|D|D|D|D|D1.900000|D|D|D|D0|D|D|C64]
```

Beginning of connection to system
Unlock Inbound and Outbound list

```
<[:BA|MUI|CE3]
<[:BA|MUO|CE9]
>[:AB|N0|D|D|D|D|D|D|D|D0|D1|D1|D1|D|D|D0|D|D|D23206|D|D|D|D|D1.860000|D|D|D|D0|D|D|C23]
```

Set up lists of inbound and outbound shared variables
Controlled from OCU to Vehicle

List OCU to Vehicle

<[:BA|N0|T0|Qvideo_mux1|T0|Qveh_start|T0|Qveh_enable|T0|Qveh_motion|T2|Qjoy_steer|T0|Qveh_steer|T0|Qveh_brake|T0|Qveh_throttle|T0|Qveh_shift|Qveh_clutch|CC9]

List Vehicle to OCU

<[:BA|L0|T2|QAutoGpsHead|T2|QAutoGpsLat|T2|QAutoGpsLon|T2|QAutoGpsVel|T0|QAutoGpsReferences|T0|QAutoGpsQuality|T2|QAutoGpsStamp|T0|Qpause|T0|Qkey_on|T0|Qeng_enb|T0|Qpe7|T0|Qbbrake|T0|Qfbrake|T2|Qsteerang|T0|Qthrottle|Qsvp_enb_eng|T0|Qserialpck|T0|Qveh_gear|T0|Qveh_state|T0|Qveh_estop|T0|Qveh_obstacle|T2|Qvehicle_bat|T4|Qrun_composite|T4|QVehicleName|T0|QHealth_total|T0|Qrpm|T0|Qfuel|T2|Qsys_thd|C52]

Write all lists to permanent memory
Lock all lists

<[:BA|MW|C9C]
<[:BA|ML|C91]
>[:AB|N0|Rvideo_mux1|D0|Rveh_start|D0|Rveh_enable|D0|Rveh_motion|D0|Rjoy_steer|D0|Rveh_steer|D0|Rveh_brake|D100|Rveh_throttle|D0|Rveh_shift|D0|Rveh_clutch|D0|C92]
>[:AB|N0|D0.000000|D40.599864|D-111.905448|D0.824760|D5|D70|D212913.000000|D0|D1|D1|D1|D|D|D0|D|D|D23207|D|D|D|D0|D1.850000|D|D|D|D0|D|D|C8D]
>[:AB|N0|D0.000000|D40.599864|D-111.905448|D0.654240|D5|D70|D212913.203000|D0|D1|D1|D1|D|D|D0|D|D1|D23208|D|D|D|D|D1.810000|D|D|D100|D0|D0|D85|CB8]
>[:AB|N0|D|D|D|D|D|D|D|D0|D1|D1|D1|D|D|D0|D|D|D23209|D|D|D|D|D1.890000|D|D|D|D0|D|D|C29]
>[:AB|N0|D0.000000|D40.599864|D-111.905448|D0.227360|D5|D70|D212913.390000|D0|D1|D1|D1|D|D|D0|D|D|D23210|D|D|D|D|D1.900000|D|D|D|D0|D|D|C58]

Connection established
Note ping/pong

<[:BA|P0|C78]
>[:AB|G0|C6F]
>[:AB|N0|D0.000000|D40.599864|D-111.905448|D0.027840|D5|D70|D212913.609000|D0|D1|D1|D1|D|D|D0|D|D|D23211|D|D|D|D|D1.840000|D|D|D|D0|D|D|C60]
>[:AB|N0|D0.000000|D40.599864|D-111.905448|D0.095120|D5|D70|D212913.796000|D0|D1|D1|D1|D100|D0|D0|D0|D1|D23213|D1|D0|D0|D0|D1.860000|D|DVEH_SN104_BARBRA|D100|D0|D0|D85|CF9]
>[:AB|N0|D0.000000|D40.599864|D-111.905448|D0.119480|D5|D70|D212914.000000|D0|D1|D1|D1|D|D|D0|D|D1|D23214|D|D|D|D|D1.910000|D|D|D100|D0|D0|D85|CB4]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D0.000000|D40.599864|D-111.905448|D0.371200|D5|D70|D212914.203000|D0|D1|D1|D1|D|D|D0|D|D|D23216|D|D|D|D|D1.910000|D|D|D|D0|D|D|C52]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D0.000000|D40.599864|D-111.905448|D0.764440|D5|D70|D212914.390000|D0|D1|D1|D1|D|D|D0|D|D|D23217|D|D|D|D|D1.870000|D|D|D|D0|D|D|C6B]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D0|D0|D0|D0|D0|D100|D0|D0|D|C07]
>[:AB|G0|C6F]
>[:AB|N0|D|D|D|D|D|D|D|D0|D1|D1|D1|D|D|D0|D|D|D23271|D|D|D|D|D1.950000|D|D|D|D0|D|D|C25]
<[:BA|N0|D|D|D1|D|D|D|D|D|D|D|C27]

Start Teleoperation

<[:BA|EServoPod|D0,TELEOP START|C02]

>[:AB|N0|D0.000000|D40.599864|D-111.905479|D0.950040|D5|D70|D212921.796000|D0|D1|D1|D1|D100|D|D0|D0|D|D23272|D|D|D|D|D1.840000|D|D|D|D0|D|D|C2F]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D1|D0|D0|D0|D100|D0|D0|D|C08]

<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|G0|C6F]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D0.000000|D40.599864|D-
111.905479|D0.609000|D5|D70|D212923.000000|D0|D1|D1|D1|D|D|D0|D|D|D23282|D|D|D|D0|D1.800000|D|D|D|D0|D|D|C84]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D0.000000|D40.599864|D-
111.905479|D0.973240|D5|D70|D212923.203000|D0|D1|D1|D1|D|D|D0|D|D1|D23283|D|D|D|D|D1.930000|D|D|D100|D0|D0|D85|CC7]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D270.310028|D40.599864|D-
111.905479|D1.249320|D5|D70|D212923.390000|D0|D1|D1|D1|D|D|D0|D|D|D23285|D|D|D|D|D1.900000|D|D|D|D0|D|D|CE1]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D1|D0|D0|D0|D100|D0|D0|D|C08]
>[:AB|G0|C6F]
>[:AB|N0|D272.370025|D40.599864|D-
111.905479|D1.329360|D5|D70|D212923.609000|D0|D1|D1|D1|D|D|D0|D|D|D23286|D|D|D|D|D1.950000|D|D|D|D0|D|D|CF2]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D|D|D|D|D|D|D|D0|D1|D1|D1|D0|D|D0|D992|D|D25492|D|D|D|D|D2.050000|D|D|D|D3160|D|D|C92]

Full Throttle

<[:BA|N0|D|D|D|D|D|D|D|D1000|D|D|CB7]

<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D268.290008|D40.599834|D-
111.905479|D1.851360|D5|D70|D213420.796000|D0|D1|D1|D1|D0|D|D0|D1000|D|D25493|D|D|D|D|D2.030000|D|D|D|D3136|D|D|C7C]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D268.899993|D40.599834|D-
111.905479|D1.641400|D5|D70|D213421.000000|D0|D1|D1|D1|D0|D0|D0|D1000|D1|D25494|D1|D0|D0|D0|D2.010000|D|DVEH_SN104_BARBRA|D100|D3112|
D0|D85|C45]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D1|D1|D0|D0|D0|D8|D0|D|CB0]
<[:BA|N0|D|D|D|D|D|D|D87|D0|D|D|C95]
<[:BA|N0|D|D|D|D|D|D|D100|D|D|D|C87]
>[:AB|G0|C6F]
>[:AB|N0|D269.290008|D40.599834|D-
111.905479|D1.532360|D5|D70|D213421.203000|D0|D1|D1|D1|D100|D|D0|D0|D1|D25496|D|D|D|D|D2.020000|D|D|D100|D0|D0|D85|CFD]
<[:BA|N0|D|D|D|D|D|D|D48|D|D|D|C62]
>[:AB|N0|D268.899993|D40.599834|D-
111.905479|D1.328200|D5|D70|D213421.390000|D0|D1|D1|D1|D48|D|D0|D0|D|D25497|D|D|D|D|D1.970000|D|D|D|D3136|D|D|C42]
<[:BA|N0|D|D|D|D|D|D|D0|D16|D|D|C8D]
>[:AB|N0|D0.000000|D40.599834|D-
111.905479|D0.941920|D5|D70|D213421.609000|D0|D1|D1|D1|D0|D|D0|D16|D|D25498|D|D|D|D|D1.900000|D|D|D|D3176|D|D|CA8]
<[:BA|N0|D|D|D|D|D-450|D|D|D|D|D|CBC]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D|D|D|D|D|D|D|D0|D1|D1|D1|D0|D|D0|D16|D|D25499|D|D|D|D|D2.090000|D|D|D|D3128|D|D|C64]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D0.000000|D40.599834|D-
111.905479|D0.627560|D5|D70|D213421.796000|D0|D1|D1|D1|D0|D|D0|D16|D|D25501|D|D|D|D0|D2.020000|D|D|D|D3144|D|D|CC6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]

Steer Left

<[:BA|N0|D|D|D|D|D-359|D|D|D|D|D|CC4]

>[:AB|N0|D0.000000|D40.599834|D-
111.905479|D0.261000|D5|D70|D213422.000000|D0|D1|D1|D1|D0|D|D0|D16|D1|D25502|D|D|D|D|D2.110000|D|D|D100|D3160|D0|D85|CCE]

```
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D1|D1|D0|D0|D0|D16|D0|D|CDF]
>[:AB|G0|C6F]
>[:AB|N0|D|D|D|D|D|D|D|D0|D1|D1|D1|D0|D|D-9|D16|D|D25503|D|D|D|D|D2.090000|D|D|D|D3024|D|D|C87]
```


Steer Right


```
<[:BA|N0|D|D|D|D|D350|D|D|D|D|D|C8E]
<[:BA|N0|D|D|D|D|D450|D|D|D|D|D|C8F]
>[:AB|N0|D0.000000|D40.599834|D-111.905479|D0.078880|D5|D70|D213422.203000|D0|D1|D1|D1|D0|D|D-
43|D16|D|D25504|D|D|D|D|D2.050000|D|D|D|D3144|D|D|CF5]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D0.000000|D40.599826|D-111.905448|D0.497640|D5|D70|D213422.609000|D0|D1|D1|D1|D0|D|D-
102|D16|D|D25505|D|D|D|D|D2.050000|D|D|D|D0|D|D|C8C]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D266.060028|D40.599826|D-
111.905479|D2.258520|D5|D70|D213423.609000|D0|D1|D1|D1|D0|D|D37|D16|D|D25511|D|D|D|D|D2.000000|D|D|D|D3064|D|D|C48]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D-380|D|D|D|D|D|CBE]
>[:AB|N0|D264.730010|D40.599826|D-
111.905479|D2.625080|D5|D70|D213423.796000|D0|D1|D1|D1|D0|D|D126|D16|D|D25512|D|D|D|D0|D2.130000|D|D|D|D3088|D|D|CB1]
<[:BA|N0|D|D|D|D|D11|D|D|D|D|D|C58]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D1|D1|D243|D0|D79|D0|D0|D|C51]
>[:AB|G0|C6F]
>[:AB|N0|D265.439971|D40.599826|D-
111.905479|D3.163320|D5|D70|D213424.000000|D0|D1|D1|D1|D79|D|D157|D0|D1|D25513|D|D|D|D|D2.100000|D|D|D100|D3152|D0|D85|CE0]
```


Steer Right with full brake


```
<[:BA|N0|D|D|D|D|D332|D|D100|D|D|D|C1F]

<[:BA|N0|D|D|D|D|D429|D|D|D|D|D|C95]
<[:BA|N0|D|D|D|D|D450|D|D75|D|D|D|CFB]
<[:BA|N0|D|D|D|D|D|D|D0|D8|D|D|C5E]
>[:AB|N0|D265.649993|D40.599826|D-
111.905479|D3.221320|D5|D70|D213424.390000|D0|D1|D1|D1|D0|D|D117|D8|D|D25515|D|D|D|D|D2.040000|D|D|D|D3112|D|D|C54]
```


Lots of Teleop Action


```
<[:BA|N0|D|D|D|D|D146|D|D|D805|D|D|C2E]
<[:BA|N0|D|D|D|D|D-98|D|D|D875|D|D|C38]
<[:BA|N0|D|D|D|D|D-450|D|D78|D0|D|D|C5B]
<[:BA|N0|D|D|D|D|D|D|D87|D|D|D|C65]
<[:BA|N0|D|D|D|D|D-408|D|D100|D|D|D|C50]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D1|D1|D-11|D0|D100|D0|D0|D|C68]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D236|D|D59|D|D|D|CFF]
>[:AB|G0|C6F]
>[:AB|N0|D265.920013|D40.599826|D-
111.905479|D2.137880|D5|D70|D213425.203000|D0|D1|D1|D1|D100|D|D7|D0|D|D25517|D|D|D|D|D1.970000|D|D|D|D0|D|D|CB2]
<[:BA|N0|D|D|D|D|D389|D|D0|D8|D|D|C02]
<[:BA|N0|D|D|D|D|D0|D|D|D203|D|D|CBB]
>[:AB|N0|D265.529998|D40.599826|D-
111.905479|D1.868760|D5|D70|D213425.390000|D0|D1|D1|D1|D0|D|D3|D203|D|D25519|D|D|D|D|D1.900000|D|D|D|D3104|D|D|C6E]
<[:BA|N0|D|D|D|D|D-232|D|D|D711|D|D|C53]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D|D|D|D|D|D|D|D0|D1|D1|D1|D|D|D7|D|D|D25552|D|D|D|D|D2.090000|D|D|D|D0|D|D|C2C]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
```

```
>[:AB|N0|D266.949981|D40.599826|D-
111.905525|D1.869920|D5|D70|D213430.796000|D0|D1|D1|D1|D|D|D7|D|D|D25553|D|D|D|D|D2.260000|D|D|D|D0|D|D|C0F]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
```

Stop Teloperation

```
<[:BA|EServoPod|D0,TELEOP STOP|CBA]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
>[:AB|N0|D267.389984|D40.599826|D-
111.905555|D1.742320|D5|D70|D213431.000000|D0|D1|D1|D1|D|D|D7|D|D|D25554|D|D|D|D0|D2.050000|D|D|D|D0|D|D|C1D]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D1|D0|D0|D0|D100|D0|D0|D|C08]
>[:AB|G0|C6F]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
```

Back to transmit only comms

```
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D1|D0|D0|D0|D100|D0|D0|D|C08]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|P0|C78]
<[:BA|N0|D0|D0|D1|D0|D0|D0|D100|D0|D0|D|C08]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
<[:BA|N0|D|D|D|D|D|D|D|D|D|D|CF6]
```